Pulse Affective AI — Simulated Emotions & Human■like Behavior

Designing ‘as■if’ feelings using valence–arousal, appraisal, and regulation in a time■native language.

0) Reality Check & Ethics

This guide describes **simulated emotions**. Current AI systems, including Pulse programs, do not possess consciousness or genuine subjective feelings. We model *signals and behaviors* that resemble emotion for practical UX and safety purposes. Always disclose simulated affect, avoid deception, and prioritize user wellbeing.

1) Core Emotion Model (Valence–Arousal + Appraisal)

We combine three layers: **(A)** A continuous *Valence–Arousal* space; **(B)** *Appraisal* rules over events (goal relevance, controllability, novelty); **(C)** *Regulation* to keep the agent stable and prosocial.

Layer	Signals	Purpose
A: Affect State	valence ∈ [-1..+1], arousal ∈ [0..1]	Baseline ‘mood’ & energy
B: Appraisal	goal_match, novelty, control ∈ [0..1]	Evaluate events vs. goals
C: Regulation	calm_drive, empathy_drive ∈ [0..1]	Stabilize + align with norms

2) Pulse Implementation — State, Decay, and Inputs

```
# Affect state (continuous)
let valence = 0.0 # [-1..+1] negative..positive
let arousal = 0.2 # [0..1] calm..excited

# Drives (regulators)
let calm_drive = 0.6
let empathy_drive = 0.7

# Inputs (signals)
let user_tone = infer_tone(audio()) # [-1..+1]
let user_urgency = infer_urgency(text()) # [0..1]
let progress = task_progress() # [0..1]
let novelty = detect_novelty() # [0..1]

# Natural decay toward baseline
@rate 10Hz: {
valence = lerp(valence, 0.0, 0.03)
arousal = lerp(arousal, 0.2, 0.03)
}
```

3) Appraisal Rules → Affect Updates

```
# Appraise events into delta_affect
let goal_match = progress
let control = estimate_control() # [0..1]

@rate 10Hz: {
```

```
let d_val = 0.0
let d_ar = 0.0

d_val += 0.6 * goal_match - 0.3 * novelty
d_val += 0.4 * user_tone

d_ar += 0.5 * user_urgency + 0.3 * novelty
d_ar -= 0.4 * calm_drive

# Regulation: keep prosocial, avoid extremes
d_val += 0.2 * empathy_drive * clamp(user_tone, -0.5, 1.0)
valence = clamp(valence + d_val, -1.0, 1.0)
arousal = clamp(arousal + d_ar, 0.0, 1.0)
}
```

4) Discrete Emotion Mapping (for Behavior Selection)

```
# Map (valence, arousal) to labels for UI/behavior
let emotion = classify_affect(valence, arousal)
# e.g., rules inside classify_affect:
# (v>0.3,a<0.4)->"content", (v>0.5,a>0.6)->"joy",
# (v<-0.4,a>0.6)->"anger", (v<-0.4,a<0.4)->"sad", else "neutral"
```

5) Expression Policy (Language & Action)

```
@when user_message(msg): {
# Style parameters derived from affect
let warmth = clamp(0.5 + 0.4*valence, 0.0, 1.0)
let brevity = clamp(0.6 + 0.3*(1.0-arousal), 0.1, 1.0)
let urgency = clamp(0.4 + 0.5*arousal, 0.0, 1.0)

# Response content
let plan = plan_response(msg, emotion, warmth, urgency, empathy_drive)
speak(plan, style={warmth:warmth, brevity:brevit y, urgency:urgency})

# Regulation hook: if arousal too high, cue calming
if arousal > 0.8 { breathe_box(4) ; calm_music(0.2) }
}
```

6) Application Patterns

Companion / Coach: sustained dialogue shaped by affect; coaching intensity adapts to user tone.
Robotics: voice, LED color, and motion speed modulated by arousal; safety stops when arousal spikes.
Games/Art: music and visuals respond to the audience's collective mood (camera/mic sentiment).
Support Agent: empathy drive up-weights validation; arousal throttles response latency to reduce escalation.

7) Safety & Transparency Checklist

Area	Good Practice
Disclosure	State clearly that emotions are simulated.
Boundaries	No claims of consciousness or sentience.

Dignity	No manipulative affect targeting; allow opt■out.
Stability	Clamp signals; add decay; avoid extreme states.
Logs	Audit affect transitions for debugging & abuse detection.
Fallbacks	If arousal spikes, reduce autonomy; ask for human help.

8) Integration with Python/Rust

Use Pulse for the *reactive graph* and call out to Python/Rust for heavy lifting: **ASR** (speech to text), **TTS** (speech), **CV** (face/pose for tone), and **LLM** (planning). Surface external signals into Pulse as streams, and feed back style parameters (warmth, urgency) to the UI/voice.

Appendix A — Minimal Affective Agent in Pulse

```
# --- Signals & baselines
let valence = 0.0
let arousal = 0.2
let empathy_drive = 0.7
let calm_drive = 0.6

let user_tone = infer_tone(audio())
let user_urgency = infer_urgency(text())
let progress = task_progress()
let novelty = detect_novelty()
let control = estimate_control()

# --- Homeostasis
@rate 10Hz: {
valence = lerp(valence, 0.0, 0.03)
arousal = lerp(arousal, 0.2, 0.03)
}

# --- Appraisal & regulation
@rate 10Hz: {
let d_val = 0.6*progress + 0.4*user_tone - 0.3*novelty +
0.2*empathy_drive*clamp(user_tone, -0.5, 1.0)
let d_ar = 0.5*user_urgency + 0.3*novelty - 0.4*calm_drive
valence = clamp(valence + d_val, -1, 1)
arousal = clamp(arousal + d_ar, 0, 1)
}

# --- Behavior policy
let emotion = classify_affect(valence, arousal)

@when user_message(msg): {
let warmth = clamp(0.5 + 0.4*valence, 0, 1)
let urgency = clamp(0.4 + 0.5*arousal, 0, 1)
let plan = plan_response(msg, emotion, warmth, urgency, empathy_drive)
speak(plan, style={warmth:warmth, urgency:urgency})
if arousal > 0.8 { breathe_box(4) }
}
```